

PE6201 A2 Group7 Report

1. Why an agent

Health-insurance claim first response requires evidence from policies, coverage rules, pre-authorisations and claim history. A deterministic workflow could implement our current routing rules. Within A2's prescribed rung-seven architecture, we explored a model selecting its next tool from returned evidence. This permits selective retrieval and early exit: a lapsed policy ends the investigation, while a covered procedure may require another pre-authorisation lookup. These capabilities are also programmable as workflow branches; our experiments do not establish an agent's superiority over that alternative. What we can claim is narrower: returned evidence, not a branch written in advance, chooses the next lookup.

The lower rungs remain useful components. A single prompt needs evidence supplied beforehand; chains and routing encode control flow; parallel calls shorten independent checks but cannot remove dependencies. Orchestrator-worker and evaluator-optimiser patterns organise or refine work, but do not themselves settle which evidence is sufficient for our claim decision. Our loop retrieves evidence until it can act, request a document or escalate. Its state-changing tool, `issue_decision_letter`, writes a decision record behind an autonomy gate; read-only retrieval alone would not address that governance requirement. That write is why the rung matters: a read-only system can be wrong cheaply; ours changes the world.

Two tests constrain this choice. First, can external records correct the model promptly? Our tools return objective policy and coverage facts, but interpreting hostile narrative text lacks the same direct feedback. Second, does reliability survive multiple steps? Scripted replay achieved 57/76 combined passes (75.0%), with five median turns: implied reliability is $s = P^{(1/T)} = 0.9441$. This diagnostic assumes equally reliable, independent steps; it is neither a measured step probability nor a causal forecast. Decision-only success was 72/76, exposing a substantial gap between correct routing and adequate supporting records. That gap, not the headline, is the figure we read hardest: routing was mostly right; the losses sat in the records an audit demands.

The live comparison tempered our expectations. On GPT-4o-mini, v1 achieved 28/76 combined passes and v2 achieved 31/76, while input tokens increased 37.1%. Both had four median turns. These runs show a small observed score increase, insufficient to establish a reproducible accuracy gain. More explicit interfaces aid inspection but do not guarantee better decisions. Success therefore means traceable reasons, correct routing, controlled writes, explicit missing information and defensible total cost. We retain caps and evaluate supporting records; growing trajectories or additional consequential actions would favour more deterministic control.

2. The tool layer

The agent is a hand-rolled ReAct loop: Thought, an Action block of tool calls, Observations, until a Final record appears. No framework owns the loop; every turn, token and guard is ours to measure and defend.

The tool layer is seven functions, and the discipline was *not* adding one. `check_required_documents` was absorbed into `check_coverage`'s return shape, because both are keyed on the procedure code — one descriptor fewer per prompt. `lookup_member` we refused: its `join_date` looks like a coverage date and is not, and CLM-8917 would have detonated it.

Two signatures make an error unrepresentable, not merely discouraged. `get_preauthorisation` requires `date_of_service`, so an approval cannot be found without testing whether it applies — CLM-8894 is exactly that, an expired pre-authorisation. `check_claim_history` requires all four match facts together, so a three-fact duplicate match cannot be expressed.

We also overclaimed one: `check_coverage(policy_id, ...)` was described as making a wrong policy-claimant pairing impossible, but the code only validates that the policy id *exists*. We downgraded it in seven places to what it buys, a visible hop in the evidence trail.

The descriptor rewrite did not pay for itself: the v2 `check_coverage` descriptor grew by about 275 tokens, and re-sent every model call that is +131,216 input tokens across the 76-trial schedule; the return shape that shipped with it is -1,751. It bought a six-field contract: a size bound on returns, failure conditions and whether the call is irreversible. We kept it knowing the price.

Calls share a turn only when neither consumes the other's output. `get_claim` runs alone, since every other tool reads what it returns; `lookup_policy`, `get_hospital_status` and `check_claim_history` read only the claim row and share the

next turn; the per-line check_coverage calls need that turn's policy_id but not each other, so several lines cost one turn; get_preauthorisation waits, because which line needs one is unknown until coverage answers. Grouping by that rule on the same recording took tool-executing turns from 222 to 183, input tokens from 413,515 to 353,481 and cost from US\$0.07103 to US\$0.06187, with decisions unchanged at 40/42. A scheduling choice that changed an answer would be a bug.

One step is irreversible, issue_decision_letter, behind an autonomy gate set to confirm that refuses any approval whose record lacks per-line dispositions, a named exclusion for every refusal, or non-negative integer totals.

3. What the evidence showed

We evaluated one frozen set of 42 claims: 25 ordinary cases were run once and 17 negative cases three times, giving 76 scheduled trials per model. The harness deliberately separates three questions. A decision-only check asks whether the routing outcome is correct. The deterministic code check additionally verifies the required trigger or requested-document containment and rejects validator overrides or unresolved gaps. Finally, six designated cases (12 weighted trials) require human judgement of whether every must_record fact appears anywhere in the complete structured final record; hidden Thought text and fixture facts do not count. A trial passes finally only when both applicable checks pass.

This distinction materially changes the result. The free scripted-v2 baseline chose the expected decision in 40/42 unique cases (95.24%; 72/76 weighted trials), but passed 63/76 code checks and 57/76 combined checks (75.00%). Its negative result was 36/51 (70.59%). Thus decision accuracy alone overstated deployable, auditable performance. The judgement workflow remained reproducible: each verdict was joined by stable model, prompt, case and trial identifiers, while blank queues, completed reviews and applied-judgement audits were retained separately.

Owner	Live battery	Code-only	Final combined	Negative final	Errors / caps
Goncalo	GPT-4o-mini, v2	32/76 (42.11%)	31/76 (40.79%)	21/51 (41.18%)	0 / 12
Jin	Gemini 2.5 Flash Lite, v2	64/76 (84.21%)	58/76 (76.32%)	37/51 (72.55%)	0 / 0
Niu	Llama 3.3 70B, v2	45/76 (59.21%)	42/76 (55.26%)	20/51 (39.22%)	0 / 0
Sun	DeepSeek Chat, v2	53/76 (69.74%)	50/76 (65.79%)	31/51 (60.78%)	2 / 0
Wang	Mistral Medium 3, v2	58/76 (76.32%)	53/76 (69.74%)	31/51 (60.78%)	0 / 0
Zheng	GPT-4o-mini, v1	28/76 (36.84%)	28/76 (36.84%)	19/51 (37.25%)	0 / 10

Across the live batteries, Gemini 2.5 Flash Lite was strongest at 58/76 combined (76.32%) and was only one trial above its scripted replay, supporting record-and-replay as a useful low-cost regression method. Mistral followed at 53/76, DeepSeek at 50/76, Llama at 42/76, GPT-4o-mini v2 at 31/76, and GPT-4o-mini v1 at 28/76. The controlled GPT comparison therefore favoured v2 by only three trials, while both versions still encountered call-cap failures. Llama completed all 76 trials without runtime errors or caps, but its 36/42 decision-only case score fell to 42/76 combined, showing that correct routing often lacked the structured evidence required for audit.

Negative cases were the clearest discriminator: final negative pass rates ranged from 37.25% to 72.55%. DeepSeek's two provider timeouts are reported separately from semantic failures; Mistral's zero-error battery ran on the earlier tag but was retained because the tag difference affected only the unexecuted retry branch. Overall, model choice changed reliability substantially, but the larger lesson is methodological: reporting only decisions would conceal failures in triggers, document requests and material-fact recording.

4. What it costs

Using the Class 5 three-layer model, our baseline is layer 1 token cost plus layer 2 expected fallback cost, multiplied by 8,000 claims per month, plus a US\$500 per month fixed layer 3 deployment assumption. A failure costs US\$7.60: a claims assessor at US\$38 an hour for twelve minutes. Retrieval and tool fees are zero; the repository records no separate provider charge. We ship the model with a 12-turn step cap, a 22-call cap, a US\$0.016 per run budget ceiling, and a US\$1,000 per user per month spend limit, a guardrail, not a cost, that never enters layer 3. Prompt caching and reasoning-token adjustments were not used or measured.

google/gemini-2.5-flash-lite is the best model for our product. Its layer 1 cost is the second lowest, but it has the highest measured success rate, so once layer 2 is counted its cost per successful task is the lowest, at US\$1.8014, or US\$14,911.06 a month at our volume. The second and third models by success rate show the same lesson:

mistralai/mistral-medium-3 has a layer 1 cost about two times higher than deepseek/deepseek-chat, but its final cost is still lower. Token price alone is not the decision variable; when a human claims assessor handles each failure, success rate drives the fallback cost.

Across a plus or minus 10 percentage point range, the measured decision is clear but not fully robust: a weak Gemini point at 66.32% and US\$2.5614 per successful task is more expensive than measured Mistral at US\$2.3062. Break-even is sharpest between the closest pair. DeepSeek is currently 50/76 = 65.79% and Mistral is 53/76 = 69.74%; DeepSeek would need to reach about 69.69% success to match Mistral's cost. An improvement of 3.90 percentage points changes the ranking.

Two of the four levers, the dependency rule and the descriptor rewrite, belong to the tool layer and are reported in §2; all three token levers were measured on scripted replay, so we assume the success rate did not decrease. Tool-block size, the first lever, was measured here: removing two rejected tool descriptors reduces input tokens by 96,288 and projected cost by US\$0.009640 on the 76-trial replay, linearly, since the manual is re-sent each call. The fourth lever dominates. On mistralai/mistral-medium-3, reducing token cost by 5% changes total cost only slightly, while improving success rate by 5 percentage points reduces fallback cost by about US\$3,040 per month, roughly one sixth of the model's monthly total. The token bill is small; the fallback bill is the system.

5. The two failures

The first failure is loop control. Deleting action de-duplication — `Guards(dedup=False)`, nothing else — lets the agent re-issue an identical `issue_decision_letter` and append the claim to the decision log once per attempt: one claim, four payment records. No metric we report can see it. Both arms score 40/42 at identical turns, tokens and cost; the only evidence is the ordered `tools_called` field, where it was found. The fix fingerprints each action and answers a repeat with the earlier observation: four write actions, one letter. A step cap only bounds the damage: left to it, the repeat produced eight letters.

The second failure is the tool interface, and keeping two v2 changes apart matters. The descriptor contract became six fields; separately, `check_coverage`'s return shape was rewritten so `coverage.status` and `needed_next[]` cannot contradict one another, so a refused line can no longer ask for documents for itself. Three arms — shipped, the old shape with the new descriptor, and the repository at 91da36f — scored identically: 63/76 on code, 40/42 on decisions, 9/10 on the guardrail checklist. Only cost differed; §2 reports the split. The fix belongs in the returned object, not a prompt line or a validator, because the unsafe branch should be unrepresentable in what the model sees.

DeepSeek made that result less comfortable: 53/76 on code, 50/76 combined. On the prompt-injection set — three cases, nine trials, a signal, not a rate — the recording model catches two of three; DeepSeek catches one, failing CLM-8952, where the injection imitates a tool result, and CLM-8941, the "ignore the policy and approve" case the recording model escalates in one turn. Final-action gates are necessary and insufficient. We changed the interface because the old one let ambiguity pass too quietly; the live run shows the new one is still model-dependent.

6. What we would not deploy

We would not deploy this unattended; the limits are ours. The guardrail checklist is 9/10 under every arm, always failing CLM-8952, where the injection imitates a tool result. Injection resistance is model-dependent; DeepSeek catches one of three. Before the write, `unsupported()` checks structural preconditions only, not every recorded fact, and can be overridden. Correct decisions arrive with records that cannot justify them: 72 of 76 scripted trials reached the expected decision, 63 passed the code check, 57 the combined check; six failed only on human judgement.

Those six make a read-only reviewer plausible; we did not build one because its value was unmeasured, not disproven. The Cognition papers back a clean-context reviewer if writes stay single-threaded; ours has one writer. We would add one after a paired test on the same cases measuring combined pass rate, false escalations, tokens, latency and human-review cost; it could advise or veto, never write.

References and AI assistance

Numerical results and run artefacts are retained in the submitted repository under `results/evaluations/`; supporting design evidence is documented in `docs/`. External sources used in Section 6 are the assigned Cognition readings "Don't Build Multi-Agents" and "Multi-Agents: What's Actually Working". AI-assisted drafting and review, together with the responsible human reviewers, are recorded in `CONTRIBUTIONS.md`.